

Práctica 4 - Servidor SSH

1. Servidor sin autenticación con par de claves pública/privada en Servicios.

1.1 Configuración del servidor SSH

Para configurar este servidor, hemos utilizado la imagen de docker **linuxserver/openssh-server** que trae el entorno preparado para **SSH** y nos permite **no tener que incluir un dockerfile** para la configuración interna en el contenedor. Aún así necesitaremos pasar ciertas **variables de entorno** y **archivos de configuración**. A continuación está el dockerfile.

```
# /stacks/services/compose.yml
services:
  ssh_manager:
    image: linuxserver/openssh-server:latest
    container_name: service-ssh-server
    environment:
      - PUID=1000
      - PGID=1000
      - TZ=Etc/UTC
      - USER_NAME=admin_user
      - SUDO_ACCESS=true
      - LOG_STDOUT=true
    networks:
      services_net:
        ipv4_address: 172.20.0.40
    volumes:
      - ./ssh/config:/config
      - ssh_host_keys:/config/ssh_host_keys
    ports:
      - "2222:2222"
    restart: always
```

También tenemos el archivo de configuración del servidor, **sshd_config**, en el que incluimos los requisitos pedidos por el enunciado.

```
# /stacks/services/ssh/config/sshd/sshd_config

# Basic Hardening
Port 2222
Protocol 2
HostKey /etc/ssh/ssh_host_rsa_key
HostKey /etc/ssh/ssh_host_ed25519_key

# Authentication

## Impide login directo como root – usar cuentas de usuario con sudo.
PermitRootLogin no
StrictModes no
## Habilita autenticación por clave pública.
PubkeyAuthentication yes
## Archivo donde buscar las claves autorizadas (aquí montado en /config para
persistencia).
AuthorizedKeysFile /config/.ssh/authorized_keys
## Deshabilita autenticación por contraseña; usar sólo claves públicas.
PasswordAuthentication no
## Asegura que cuentas sin contraseña no puedan autenticarse.
PermitEmptyPasswords no
ChallengeResponseAuthentication no
UsePAM no

# Authorization
## Restringe el acceso SSH a usuarios listados (ej. 'admin_user').
AllowUsers admin_user
X11Forwarding no
PrintMotd no

# Logging
SyslogFacility AUTH
LogLevel VERBOSE
```

1.2 Despliegue del servidor SSH

Antes de poder desplegar el servicio, se necesitan generar las **ssh_host_keys**, que son las claves públicas y privadas que **identifican el servidor** ante los clientes.

Esto se tendría que hacer de forma manual usando comandos, pero nosotros hemos creado un **script** para poder generarlas siempre que las necesitemos llamado **recreate-ssh-secrets.sh** y las guarda en la carpeta **/ssh_host_keys**.

Este script aparte de generar lo mencionado anteriormente crea un fichero **authorized_keys** a partir del **authorized_keys.example**, este archivo es necesario pq le dice al servidor que clientes pueden acceder, a partir de las **claves públicas** de estos.

Esta es la salida del script:

```
symbiontz@DESKTOP-NIUV50S:/mnt/c/Users/symbiontz/projects/AS-2025$ ./scripts/recreate-ssh-secrets.sh
[2026-04-28T17:38:43Z] CREADA: ssh_host_rsa_key
[2026-04-28T17:38:43Z] Fingerprint ssh_host_rsa_key.pub: 4096 SHA256:bSzwpKmw8ygk+U5C5hpNufcXHweoJP0CgFEnARvfi8M symbiontz@DESKTOP-NIUV50S (RSA)
[2026-04-28T17:38:43Z] CREADA: ssh_host_ecdsa_key
[2026-04-28T17:38:43Z] Fingerprint ssh_host_ecdsa_key.pub: 256 SHA256:MARZPqdC9ySunTyHSNz13Lur6yanuHrQHjF5pP6TK+o symbiontz@DESKTOP-NIUV50S (ECDSA)
[2026-04-28T17:38:43Z] CREADA: ssh_host_ed25519_key
[2026-04-28T17:38:43Z] Fingerprint ssh_host_ed25519_key.pub: 256 SHA256:UCoc7k0//Wfdu0qKn3Q/8Udsj0x7BXttQTrcasa/qDk symbiontz@DESKTOP-NIUV50S (ED25519)
[2026-04-28T17:38:43Z] Sincronizando host keys hacia el volumen Docker: services_ssh_host_keys
[2026-04-28T17:38:43Z] Host keys disponibles en el volumen: services_ssh_host_keys
[2026-04-28T17:38:43Z] EXISTE: authorized_keys; no se sobrescribe
[2026-04-28T17:38:43Z] Listo. Secretos SSH preparados en el volumen: services_ssh_host_keys
```

También antes de poder entrar necesitamos tener unas **claves generadas** y que estén en el fichero **authorized_keys**. Hemos creado tambien un **script** para esto llamado **generate-share-ssh-key.sh**

Esta es la salida:

```
symbiontz@DESKTOP-NIUV50S:/mnt/c/Users/symbiontz/projects/AS-2025$ ./scripts/generate-share-ssh-key.sh --force
[2026-04-28T17:43:25Z] Sobrescribiendo clave privada existente en: /home/symbiontz/.ssh/id_ed25519_as2025
[2026-04-28T17:43:25Z] Backup: existing authorized_keys -> /mnt/c/Users/symbiontz/projects/AS-2025/stacks/services/ssh/config/.ssh/authorized_keys.bak.20260428T174325Z
[2026-04-28T17:43:25Z] Generando par de claves en: /home/symbiontz/.ssh/id_ed25519_as2025
[2026-04-28T17:43:25Z] Instalando clave pública en: /mnt/c/Users/symbiontz/projects/AS-2025/stacks/services/ssh/config/.ssh/authorized_keys
[2026-04-28T17:43:25Z] Fingerprint clave pública: 256 SHA256:XZF9uduXltzmSho5gLeTaTYfy/6LneAVmBEHL820r7M as2025@DESKTOP-NIUV50S (ED25519)
[2026-04-28T17:43:25Z] Hecho. Para conectarte desde este equipo (privada en /home/symbiontz/.ssh/id_ed25519_as2025):
ssh -p 2222 -i /home/symbiontz/.ssh/id_ed25519_as2025 admin_user@HOST>
[2026-04-28T17:43:25Z] Si el contenedor corre en la misma máquina, usa HOST=127.0.0.1
[2026-04-28T17:43:25Z] Nota: la clave privada NO se sube al repositorio. Manténla segura.
```

Una vez realizado, esto ya está disponible para conectarse al servidor.

1.3 Verificación del servidor SSH

```
PS C:\Users\symbiontz\projects\AS-2025> ssh -i $env:USERPROFILE\.ssh\id_ed25519_as2025 -p 2222 admin_user@localhost
d6211110f165:~$ exit
logout
Connection to localhost closed.
```

Aquí podemos ver una foto de como nos hemos podido conectar al servidor correctamente.

2. Servidor con 2FA Google Authenticator en Desarrollo.

2.1 Configuración del servicio

Para configurar este servidor, hemos utilizado un dockerfile con **ubuntu 22.04**. En este caso en el **compose.yml**, tenemos que indicar la ruta al dockerfile, y en este caso lo exponemos en el puerto 2223 para no colisionar con el otro servicio.. A continuación tenemos los archivos de docker y de configuración del servidor.

```
# stacks/development/compose.yml
services:
  ssh_2fa:
    build:
      context: ./ssh-2fa
      dockerfile: Dockerfile
    container_name: dev_ssh_2fa
    hostname: dev-ssh
    ports:
      - "2223:2222"
    networks:
      development_net:
        ipv4_address: 172.40.0.30
    volumes:
      - ./ssh-2fa/users.conf:/etc/ssh-2fa/users.conf:ro
      - ssh_2fa_secrets:/home
    restart: always
```

```
# stacks/development/ssh-2fa/Dockerfile

FROM ubuntu:22.04

ENV DEBIAN_FRONTEND=noninteractive

RUN apt-get update && \
    apt-get install -y openssh-server libpam-google-authenticator sudo
&& \
    apt-get clean && \
    rm -rf /var/lib/apt/lists/*

RUN mkdir -p /run/sshd

COPY sshd_config /etc/ssh/sshd_config
COPY pam_sshd /etc/pam.d/sshd
COPY entrypoint.sh /entrypoint.sh
```

```
# Normalizar en el caso de editar con windows
RUN sed -i 's/\r$//' /etc/ssh/sshd_config /etc/pam.d/sshd /entrypoint.sh

RUN chmod +x /entrypoint.sh

EXPOSE 2222

CMD ["/entrypoint.sh"]
```

```
# stacks/development/ssh-2fa/sshd_config

# Basic Hardening
Port 2222
Protocol 2
HostKey /etc/ssh/ssh_host_rsa_key
HostKey /etc/ssh/ssh_host_ed25519_key

# Authentication
PermitRootLogin no
StrictModes yes
PubkeyAuthentication no
PasswordAuthentication no
ChallengeResponseAuthentication yes
UsePAM yes ## Necesario para activar 2FA con Google Authenticator
AuthenticationMethods keyboard-interactive ## Para interactuar por
teclado al intentar conectarse

# Authorization
X11Forwarding no
PrintMotd no

# Logging
SyslogFacility AUTH
LogLevel VERBOSE
```

```
# stacks/development/ssh-2fa/pam_sshd

# PAM configuration for sshd with Google Authenticator

# Standard Unix authentication (password)
@include common-auth

# Google Authenticator TOTP
# nullok allows users to login without TOTP if they haven't set it up
```

```

yet (useful for initial setup/enrollment)
# We remove nullok so they MUST have a TOTP setup to login, but our
entrypoint script sets it up for them.
auth required pam_google_authenticator.so

# Standard Unix account, session, and password management
@include common-account
@include common-session
@include common-password

```

Este último archivo es muy importante para poder usar el 2FA con Google.

2.2 Despliegue del servicio

Esté contenedor es mucho más sencillo de desplegar, ya que solo necesitamos levantar el contenedor. De lo único que tenemos que preocuparnos sería de **crear usuarios aceptados por el servidor**, estos son guardados en el fichero **users.conf**, para ello hemos creado también otro script para ello, llamado **create-ssh-2fa-user.sh**, a la hora de ejecutar el script tenemos que **indicar el usuario y la contraseña de este**. Y te **devolverá un código TOTP** necesario para **activarlo en la app Google Authenticator**.

```

symbiontz@DESKTOP-NIUV50S:/mnt/c/Users/symbiontz/projects/AS-2025$ ./scripts/create-ssh-2fa-user.sh pruebadesarrollador1 Test123
Cuenta creada correctamente
- Usuario: pruebadesarrollador1
- SSH: ssh -p 2223 pruebadesarrollador1@localhost
- Secreto TOTP: FMBT2PYRIG5SPMXMFEJNJSNCA4

```

The screenshot shows the Google Authenticator app interface. At the top, there is a back arrow and the text 'Introduce los detalles del código'. Below this, there are three input fields: 'Nombre en clave' with the value 'Docker pruebadesarrollador1', 'Tu clave' with the value 'FMBT2PYRIG5SPMXMFEJNJSNCA4', and 'Tipo de clave' with a dropdown menu showing 'Basada en tiempo'. At the bottom, there is a red button labeled 'Añadir'.

NO ES POSIBLE SACAR CAPTURAS DE PANTALLA A LA APP DE GOOGLE AUTHENTICATOR PARA LOS CÓDIGOS.

Ya estaría listo para poder ingresar al servidor ssh.

2.3 Verificación del servidor ssh

A continuación mostraré la salida del comando para entrar.

```

symbiontz@DESKTOP-NIUV50S:/mnt/c/Users/symbiontz/projects/AS-2025$ ssh -p 2223 test1@localhost
(test1@localhost) Password:
(test1@localhost) Verification code:
test1@dev-ssh:~$

```